

PROGRAM

Table creation and insertion

```
CREATE TABLE Students (StudentID INT PRIMARY KEY, Name VARCHAR(30), Age INT);
CREATE TABLE Courses (CourseID INT PRIMARY KEY, CourseName VARCHAR(20));
CREATE TABLE Enrollments (StudentID INT REFERENCES Student(StudentID), CourseID INT REFERENCES Courses(CourseID));
INSERT INTO Students VALUES (1, 'Alice', 20);
INSERT INTO Courses VALUES (101, 'Database Management');
INSERT INTO Enrollments VALUES (1, 101);
```

Simple Queries

```
SELECT * FROM Students;
```

OUTPUT:

StudentID	Name	Age
1	Alice	20
2	Bob	22
3	Charlie	21
4	David	19

Nested Queries

```
SELECT Name, Age
FROM Students
WHERE Age > 20;
```

Name	Age
Bob	22
Charlie	21

```
SELECT Name
FROM Students
WHERE StudentID IN (
    SELECT StudentID
    FROM Enrollments
    WHERE CourseID = (
        SELECT CourseID
        FROM Courses
        WHERE CourseName = 'Database Management'
    )
);
```

OUTPUT:

Name

Alice

Charlie

Subqueries

```
SELECT AVG(Age) AS AverageAge
FROM Students;
```

AverageAge

20.5000

```
SELECT CourseID, CourseName
FROM Courses
WHERE CourseID IN (
    SELECT CourseID
    FROM Enrollments
    GROUP BY CourseID
    HAVING COUNT(*) > 1
);
```

OUTPUT:

CourseID

CourseName

101

Database Management

102

Algorithms

103

Web Development

```
SELECT Name, Age
FROM Students
WHERE Age > (
    SELECT AVG(Age)
    FROM Students
);
```

OUTPUT:

Name

Age

Bob

22

Charlie

21

PROGRAM

Table creation and insertion

```
CREATE TABLE Student (StudentID INT, Name VARCHAR(30), Age INT);
CREATE TABLE Courses (CourseID INT, CourseName VARCHAR(20));
CREATE TABLE Enrollments (EnrollmentID INT, StudentID INT, CourseID INT, Grade
VARCHAR(5));
```

```
INSERT INTO Student VALUES (1, 'Alice', 20);
INSERT INTO Courses VALUES (1, 'Math');
INSERT INTO Enrollments VALUES (1, 1, 1, 'A');
```

INNER JOIN

```
SELECT Student.StudentID, Student.Name, Student.Age,
       Courses.CourseID, Courses.CourseName, Enrollments.Grade
FROM Student
INNER JOIN Enrollments
    ON Student.StudentID = Enrollments.StudentID
INNER JOIN Courses
    ON Enrollments.CourseID = Courses.CourseID;
```

OUTPUT:

StudentID	Name	Age	CourseID	CourseName	Grade
2	Bob	22	1	Math	A-
1	Alice	20	1	Math	A
3	Charlie	21	2	English	A
1	Alice	20	2	English	B
3	Charlie	21	3	History	B+

LEFT JOIN

```
SELECT Student.StudentID, Student.Name, Student.Age,
       Courses.CourseID, Courses.CourseName, Enrollments.Grade
FROM Student
LEFT JOIN Enrollments
    ON Student.StudentID = Enrollments.StudentID
LEFT JOIN Courses
    ON Enrollments.CourseID = Courses.CourseID;
```

OUTPUT:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	2	English	B
1	Alice	20	1	Math	A
2	Bob	22	1	Math	A-
3	Charlie	21	2	English	A
3	Charlie	21	3	History	B+

RIGHT JOIN

```

SELECT Student.StudentID, Student.Name, Student.Age,
       Courses.CourseID, Courses.CourseName, Enrollments.Grade
FROM Courses
RIGHT JOIN Enrollments
  ON Courses.CourseID = Enrollments.CourseID
RIGHT JOIN Student
  ON Enrollments.StudentID = Student.StudentID;

```

OUTPUT:

StudentID	Name	Age	CourseID	CourseName	Grade
1	Alice	20	2	English	B
1	Alice	20	1	Math	A
2	Bob	22	1	Math	A-
3	Charlie	21	2	English	A
3	Charlie	21	3	History	B+

PROGRAM

Table creation and Insertion

```
CREATE TABLE students (student_id SERIAL PRIMARY student_name VARCHAR(student_email
VARCHAR(100));
```

```
INSERT INTO students (student_name, student_email)
VALUES ('Alice Johnson', 'alice@example.com');
INSERT INTO students (student_name, student_email)
VALUES ('Bob Smith', 'bob@example.com');
INSERT INTO students (student_name, student_email)
VALUES ('Charlie Brown', 'charlie@example.com');
```

Creating a View and DML Operations

```
CREATE VIEW student_view AS
SELECT student_id, student_name, student_email
FROM students;
```

```
INSERT INTO students (student_name, student_email)
VALUES ('Diana Prince', 'diana@example.com');
```

```
UPDATE students
SET student_email = 'new_bob@example.com'
WHERE student_name = 'Bob Smith';
```

```
DELETE FROM students
WHERE student_name = 'Charlie Brown';
```

Display view

```
SELECT * FROM student_view;
```

OUTPUT:

student_id	student_name	student_email
1	Alice Johnson	alice@example.com
2	Bob Smith	new_bob@example.com
4	Diana Prince	diana@example.com

Create a index

```
CREATE INDEX idx_student_email
ON students (student_email);
```

Display indexes

SHOW INDEX FROM students;

OUTPUT:

Table	Non_u nique	Key_n ame	Seq_in _index	Colum n_nam e	Collati on	Cardin ality	Null	Index_ type	Visible
student s	0	PRIM ARY	1	student _id	A	3	—	BTRE E	YES
student s	0	student _id	1	student _id	A	3	—	BTRE E	YES
student s	1	idx_stu dent_e mail	1	student _email	A	3	YES	BTRE E	YES

PROGRAM

Table Creation and Insertion

```
CREATE TABLE customers (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    address VARCHAR(100),  
    salary DECIMAL(10,2)  
);
```

```
INSERT INTO customers (id, name, address, salary) VALUES  
(1, 'John', 'New York', 5000.00),  
(2, 'Alice', 'Los Angeles', 6000.00),  
(3, 'Bob', 'Chicago', 4500.00),  
(4, 'David', 'Houston', 7000.00),  
(5, 'Emma', 'Boston', 5500.00);
```

Implicit Cursors:

```
DECLARE  
total_rows number(2);  
BEGIN  
UPDATE customers  
SET salary = salary + 500;  
IF sql%notfound THEN dbms_output.put_line('no customers selected');  
ELSIF sql%found THEN  
total_rows := sql%rowcount;  
dbms_output.put_line(total_rows || ' customers selected');  
END IF;  
END;  
/  
Output:
```

6 customers selected
PL/SQL procedure successfully completed.

Explicit Cursors:

```
DECLARE  
c_id customers.id%type;  
c_name customer.name%type;  
c_addr customers.address%type;  
CURSOR c_customers is SELECT id, name, address FROM customers;  
BEGIN  
OPEN c_customers;  
LOOP  
FETCH c_customers into c_id, c_name, c_addr;  
EXIT WHEN c_customers%notfound;
```

```
dbms_output.put_line(c_id || " " || c_name || " " || c_addr);  
END LOOP;  
CLOSE c_customers;  
END;  
/
```

Output:

ID	NAME	ADDRESS
1	Ramesh	Ahmadabad
2	Khilan	Delhi
3	Kaushik	Kota
4	Chaitali	Mumbai

PROGRAM

Procedure and Procedure Call

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE Sum_Numbers (
    a IN NUMBER,
    b IN NUMBER
) IS
    c NUMBER;
BEGIN
    c := a + b;
    DBMS_OUTPUT.PUT_LINE('Sum of two nos = ' || c);
END Sum_Numbers;
/
```

```
BEGIN
    Sum_Numbers(10, 20);
END;
/
```

Output:

Result

Sum of two nos = 30

Function and Function Call

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE FUNCTION Sum_Function (
    a IN NUMBER,
    b IN NUMBER
) RETURN NUMBER
IS
    c NUMBER;
BEGIN
    c := a + b;
    RETURN c;
END Sum_Function;
/
```

```
SET SERVEROUTPUT ON;
DECLARE
    result NUMBER;
BEGIN
    result := Sum_Function(5, 5);
```

```
DBMS_OUTPUT.PUT_LINE('Sum of two nos = ' || result);  
END;  
/  
Output
```

Result

Sum of two nos = 10

PROGRAM

TRIGGER ON UPDATE AND TESTING

```
CREATE OR REPLACE TRIGGER up_classd
BEFORE UPDATE ON customer
FOR EACH ROW
BEGIN
    IF UPDATING THEN
        DBMS_OUTPUT.PUT_LINE('new value is ' || :NEW.stotal);
        DBMS_OUTPUT.PUT_LINE('old value is ' || :OLD.stotal);
    END IF;
END;
/
```

```
UPDATE customer
SET stotal = 500
WHERE sid = 3;
```

OUTPUT:

```
new value is 500
old value is 900

1 row updated.
```

TRIGGER ON DELETE AND TESTING

```
CREATE OR REPLACE TRIGGER del_classb
BEFORE DELETE ON customer
FOR EACH ROW
BEGIN
    IF DELETING THEN
        DBMS_OUTPUT.PUT_LINE('row deleted');
    END IF;
END;
/
```

```
DELETE FROM customer
WHERE sid = 1;
```

OUTPUT:

```
row deleted

1 row deleted.
```

TRIGGER ON INSERT AND TESTING

```
CREATE OR REPLACE TRIGGER ins_classb
BEFORE INSERT ON classb
FOR EACH ROW
DECLARE
    InvTot EXCEPTION;
BEGIN
    IF INSERTING THEN
        IF :NEW.stotal > 1000 THEN
            RAISE InvTot;
        END IF;
    END IF;

EXCEPTION
    WHEN InvTot THEN
        RAISE_APPLICATION_ERROR(-20000, 'Total not valid');
END;
/
```

```
INSERT INTO classb VALUES (1, 'John', 'IT', 900, 'A');
SELECT * FROM classb;
OUTPUT:
```

sid	sname	sdept	stotal	grade
1	John	IT	900	A

```
INSERT INTO classb
VALUES ('6', 'jana', 'it', '20000', 'a');
```

OUTPUT:

ERROR ORA-20000: Total not valid

PROGRAM

Table creation and Insertion

```
CREATE TABLE (AuthorID INT PRIMARY KEY AUTO_INCREMENT,FirstName VARCHAR(50),LastName VARCHAR(50));
```

```
CREATE TABLE Books (BookID INT PRIMARY KEY AUTO_INCREMENT,Title VARCHAR(100),Genre VARCHAR(50),PublicationYear INT);
```

```
CREATE TABLE BookAuthors (BookID INT,AuthorID INT, PRIMARY KEY (BookID,AuthorID),FOREIGN KEY (BookID) REFERENCES Books(BookID),FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID));
```

```
CREATE TABLE Borrowers (BorrowerID INT PRIMARY KEY AUTO_INCREMENT,FirstName VARCHAR(50),LastName VARCHAR(50),MembershipDate DATE);
```

```
CREATE TABLE BorrowedBooks (BorrowerID INT,BookID INT,BorrowedDate DATE,ReturnDate DATE,PRIMARY KEY (BorrowerID, BookID),FOREIGN KEY (BorrowerID) REFERENCES Borrowers(BorrowerID),FOREIGN KEY (BookID) REFERENCES Books(BookID));
```

```
INSERT INTO Authors (FirstName, LastName)VALUES ('George', 'Orwell');
```

```
INSERT INTO Books (Title, Genre, PublicationYear)VALUES ('1984', 'Dystopian', 1949);
```

```
INSERT INTO BookAuthors (BookID, AuthorID)VALUES (1, 1);
```

```
INSERT INTO Borrowers (FirstName, LastName, MembershipDate)VALUES ('John', 'Doe', '2023-01-01');
```

```
INSERT INTO BorrowedBooks(BorrowerID, BookID, BorrowedDate, ReturnDate)VALUES(1, 1, '2023-03-01', '2023-03-15');
```

```
SELECT * FROM Authors;
OUTPUT:
```

AuthorID	FirstName	LastName
1	George	Orwell
2	Aldous	Huxley
3	J.K.	Rowling

SELECT * FROM Books;
OUTPUT:

BookID	Title	Genre	PublicationYear
1	1984	Dystopian	1949
2	Brave New World	Dystopian	1932
3	Harry Potter and the Sorcerer's Stone	Fantasy	1997

SELECT * FROM BookAuthors;
OUTPUT:

BookID	AuthorID
1	1
2	2
3	3

SELECT * FROM Borrowers;
OUTPUT:

BorrowerID	FirstName	LastName	MembershipDate
1	John	Doe	2023-01-01
2	Jane	Smith	2023-02-15

SELECT * FROM BorrowedBooks;
OUTPUT:

BorrowerID	BookID	BorrowedDate	ReturnDate
1	1	2023-03-01	2023-03-15
2	3	2023-03-05	2023-03-20

PROGRAM

```
SELECT * FROM customers;
```

ID	Name	Address	Salary
1	John	New York	5500.00
2	Alice	Los Angeles	6500.00
3	Bob	Chicago	5000.00
4	David	Houston	7500.00
5	Emma	Boston	6000.00

PRE-DEFINED EXCEPTIONS

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  c_id  customers.id%TYPE := 5;  
  c_name customers.name%TYPE;  
  c_addr customers.address%TYPE;
```

```
BEGIN
```

```
  SELECT name, address  
  INTO c_name, c_addr  
  FROM customers  
  WHERE id = c_id;
```

```
  DBMS_OUTPUT.PUT_LINE('Name: ' || c_name);  
  DBMS_OUTPUT.PUT_LINE('Address: ' || c_addr);
```

```
EXCEPTION
```

```
  WHEN NO_DATA_FOUND THEN  
    DBMS_OUTPUT.PUT_LINE('No such customer!');  
  WHEN OTHERS THEN  
    DBMS_OUTPUT.PUT_LINE('Error!');
```

```
END;
```

```
/
```

OUTPUT:

Name	Address
Emma	Boston

USER-DEFINED EXCEPTIONS

```
SET SERVEROUTPUT ON;
DECLARE
  c_id customers.id%TYPE := &cc_id;
  c_name customers.name%TYPE;
  c_addr customers.address%TYPE;

  ex_invalid_id EXCEPTION;

BEGIN
  IF c_id <= 0 THEN
    RAISE ex_invalid_id;
  ELSE
    SELECT name, address
    INTO c_name, c_addr
    FROM customers
    WHERE id = c_id;

    DBMS_OUTPUT.PUT_LINE('Name: ' || c_name);
    DBMS_OUTPUT.PUT_LINE('Address: ' || c_addr);
  END IF;

EXCEPTION
  WHEN ex_invalid_id THEN
    DBMS_OUTPUT.PUT_LINE('ID must be greater than zero!');

  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No such customer!');

  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error!');
END;
/
```

OUTPUT:

Enter value for cc_id: 3

Name	Address
Bob	Chicago

Enter value for cc_id: 0

Input ID	Result
0	ID must be greater than zero!